

Lab 1: Data Representation — Bit Manipulation

CSCI 3334 · UTRGV Fall 2026

Overview

Weight: 10% | **Out:** Wed Sep 9 | **Due:** Wed Sep 23

Implement 12 functions using only bitwise and integer operations. No loops, no conditionals, no macros. You may only use:

- Constants: `0x00-0xFF`
- Operators: `! ~ & ^ | + << >>`
- Variables and assignment
- `int` data type

Not allowed: `if`, `else`, `for`, `while`, `switch`, `?:`, function calls, macros, `long`, arrays, structs.

Important

This lab forces you to think in terms of **bits**, not values. Most functions can be solved in under 10 operations.

Setup

Get the starter code the same way you did for Lab 0 — if you already have the repo, `git pull` instead of cloning again:

```
git clone https://github.com/jpcoding/csci3334-labs.git
cd csci3334-labs/lab1
make                # compile bits.c, then check it
make check          # just the checker
make check P=negate # just one puzzle
```

All 12 puzzles are already stubbed in `bits.c`, and the file compiles before you write a line — every stub returns 2.

You'll edit `bits.c`. Each function has a comment specifying:

```
/*
 * bitXor - x^y using only ~ and &
 *   Example: bitXor(4, 5) = 1
 *   Legal ops: ~ &
 *   Max ops: 14
 *   Rating: 1
 */
int bitXor(int x, int y) {
    return 2; // Replace this
}
```

Functions (10 required + 2 bonus)

The two marked (*bonus*) are 10 and 12; the other ten are required.

Note

Start with Level 1 and 2

This goes out before two of the lectures it draws on. Levels 1 and 2 need only what you already have: `bitXor` through `isLessOrEqual` are all reachable on day one. `isTmax`, `isLessOrEqual` and `howManyBits` get easier after **Lecture 5** (Sep 14), and every `float_*` puzzle needs **Lecture 5b** (Sep 16) — a full week before the deadline. Do not save Level 1 for the end.

Level 1 — Warmup

#	Function	Description	Max Ops
1	<code>bitXor(x,y)</code>	$x \oplus y$ using only <code>~</code> and <code>&</code>	14
2	<code>tmin()</code>	Smallest two's complement integer	4
3	<code>isTmax(x)</code>	Returns 1 if <code>x</code> is the maximum two's complement integer	10
4	<code>allOddBits(x)</code>	Returns 1 if all odd-numbered bits are 1	12
5	<code>negate(x)</code>	Return <code>-x</code> without using <code>-</code>	5

Level 2 — Bit Counting

#	Function	Description	Max Ops
6	<code>isAsciiDigit(x)</code>	1 if $0x30 \leq x \leq 0x39$	15
7	<code>conditional(x,y,z)</code>	Same as <code>x ? y : z</code>	16
8	<code>isLessOrEqual(x,y)</code>	1 if <code>x <= y</code>	24

Level 3 — Float-Level

#	Function	Description	Max Ops
9	<code>float_neg(uf)</code>	Return <code>-f</code> as unsigned. If NaN, return <code>uf</code> .	10
10	<code>float_i2f(x)</code>	(<i>bonus</i>) Convert <code>int</code> to <code>float</code> bit pattern	30

Level 4 — Advanced

#	Function	Description	Max Ops
11	<code>howManyBits(x)</code>	Minimum bits needed to represent <code>x</code> in two's complement	90
12	<code>float_f2i(uf)</code>	(<i>bonus</i>) Convert <code>float</code> bit pattern to <code>int</code>	30

Hints

- `tmin()`: What does the bit pattern of `INT_MIN` look like?
- `bitXor(x,y)`: De Morgan's laws — $x \oplus y = (x \& \sim y) \mid (\sim x \& y)$. Now remove `|`.
- `negate(x)`: Two's complement negation: flip bits and add 1.
- For float puzzles: allowed ops include `if`, `while`, and `int/unsigned` casts, but **no** floating-point operations or unions.

Checking Your Work

`make check` answers both questions about every puzzle:

Is it legal? It reads each puzzle’s own comment — the `Legal ops` and `Max ops` lines — and reports the operators you used against the ones you are allowed, plus anything banned: a loop, a conditional, a constant over `0xFF`.

Is it right? Your function is called with a few hundred thousand values — `0`, `±1`, `TMin`, `TMax`, powers of two, ASCII boundaries, NaN, infinity, denormals, then twenty thousand pseudo-random ones per puzzle — and compared against the same answer computed in Python.

```
OK    negate
FAIL  isLessOrEqual
      (1, -2147483648) returned 1, expected 0
      rules: illegal operator(s): <=
```

A failure always prints the exact input that broke it. Feed that value back into your expression and trace it by hand — almost always faster than staring at the code.

Note

The two verdicts are independent, and both matter. An answer can be legal and wrong, which scores nothing. It can also be right and illegal, which costs you the operator bonus but keeps the correctness points. Get it right first.

Grading

Criterion	Points
Correctness (<code>make check</code>) — 10 required puzzles	90
Code style (comments, clarity)	10
<i>Bonus:</i> operator counts within limits (<code>make check</code>)	+10
<i>Bonus:</i> puzzles 10 and 12 (<code>float_i2f</code> , <code>float_f2i</code>)	+10

A correct solution that uses more operators than the stated limit still earns full correctness credit. The operator limit is a puzzle-golf challenge worth bonus points, not a penalty for solving the problem the straightforward way.

Submission

Upload `bits.c` to the Lab assignment on **Brightspace**. Submit the source file itself — not a screenshot, not a zip, not a link to a repo.

Your file must compile with the flags in the handout. Code that does not build scores the correctness points it earns when it does not build, which is none — check before you upload.

References

- CS:APP §2.1–2.4 (Integer representation, two’s complement, bit-level operations)
- CS:APP §2.4 (Floating point)
- Bit Twiddling Hacks